Exercise 5

In[1]:=
```
(* Function that defines parameters and performs preliminary work *)
preliminaries[] := Module[{},
                          c = 2;  (* Number of cranes *)

    numberOfDays = 90; (* The number of days to be simulated *)

                          (* Parameters for arrival and service distributions *)
                          λ = 1.25; (* Mean interarrival time in days *)

                          meanServiceTime = 1;
                          varianceOfServiceTime = 0.1;
                          leftTruncation = 0.5;
                          rightTruncation = 1.5;


                          (* Definitions *)
                          KINDARRIVAL = 1;
                          KINDDEPARTURE = 2;
                          KINDFASTDEPARTURE = 3;

                          (* Statistical Counters *)
                          totalNumberOfArrivals = 0;
                          totalAmountOfCranesBusy = 0;
                          totalWaitingTimeOfBoats = 0;
                          longestWaitingTimeOfBoats = 0;

                          (* Other parameters *)
                          eventList = {}; (* Events in the list have
    the form {timeOfOccurence, kind}, e.g. {5.2857, KINDDEPARTURE} *)
                          queue = {}; (* Customers in the
    queue have the form {time of arrival}, e.g. {3.4874} *)
                          simulationClock = 0;
                          currentNumberOfCranesBusy = 0;
                       ];
```

In[2]:=
```
(* Func draws from the Weibull distr. w/ mean λ *)
drawWeibullNumber[λ_] := -λ Log[RandomReal[]]
```

In[3]:=
```
(* Func draws from N(meanServiceTime, varianceOfServiceTime)
                    numerischer Wert
  in [lowerTruncation, upperTruncation] using Box-Muller*)
                                                  Box
drawNormalNumber[meanServiceTime_, varianceOfServiceTime_,
  leftTruncation_, rightTruncation_] := Module[{sample},
                                          Modul
  While[True,
  solange wahr
   (* Box-Muller method *)
      Box
   sample = meanServiceTime + Sqrt[varianceOfServiceTime] *
                               Quadratwurzel
      Sqrt[-2 Log[RandomReal[]]] * Sin[2 * Pi * RandomReal[]];
      Quadra··· L··· reelle Zufallszahl  Sinus  Kre··· reelle Zufallszahl
   (* Checks truncation *)
   If[leftTruncation < sample < rightTruncation,
   wenn
    Break[];
    beende Schleife
   ]
  ];

  sample
 ]
```

In[4]:=
```
(* Function that adds an event to the event list *)
   Funktion
addEventToEventList[event_] := Module[{},
                               Modul
                                    eventList = Sort[Append[eventList, event]];
                                                sorti·· hänge an
                                ];
```

In[5]:=
```
(* Function that removes the next event from the event list and returns it *)
   Funktion
removeAndReturnNextEventFromList[] := Module[{nextEvent},
                                      Modul
                                           nextEvent = First[eventList];
                                                       erstes Element
                                           eventList = Drop[eventList, 1];
                                                       entferne Element
                                           nextEvent
                                      ];
```

In[6]:=
```
(* Function that removes the next boat from the queue and returns it *)
   Funktion
removeAndReturnNextBoatFromQueue[] := Module[{nextBoat},
                                      Modul
                                           nextBoat = First[queue];
                                                      erstes Element
                                           queue = Drop[queue, 1];
                                                   entferne Element
                                           nextBoat
                                      ];
```

In[7]:=
```
(* Function that starts the
   Funktion
 simulation: schedules the first arrival to start up the event list *)
initialise[] := Module[{},
                Modul
                        firstArrivalTime = drawWeibullNumber[λ];
                        addEventToEventList[{firstArrivalTime, KINDARRIVAL}];
                ];
```

In[8]:=
```
(* Function that processes an arrival event *)
   Funktion
handleArrivalEvent[arrivalTime_] := Module[{nextArrivalTime, serviceTime},
                                    Modul
  (* Handle the arrival *)
  totalNumberOfArrivals += 1;

  If[currentNumberOfCranesBusy < c, (* one or two cranes available *)
  wenn
   (* The arriving boat can be taken into service immediately *)
   serviceTime = drawNormalNumber[meanServiceTime,
     varianceOfServiceTime, leftTruncation, rightTruncation];

   If[currentNumberOfCranesBusy == 0,
   wenn
    (* Two cranes available *)
    totalWaitingTimeOfBoats += serviceTime / 2;
    longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, serviceTime / 2];
                                größtes Element

    addEventToEventList[{simulationClock + serviceTime / 2, KINDFASTDEPARTURE}];
    currentNumberOfCranesBusy += 2;
    ,

    (* One crane available *)
    totalWaitingTimeOfBoats += serviceTime;
    longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, serviceTime];
                                größtes Element
    addEventToEventList[{simulationClock + serviceTime, KINDDEPARTURE}];
    currentNumberOfCranesBusy += 1;
   ],

   (* The arriving boat must queue *)
   queue = Append[queue, arrivalTime];
           hänge an
  ];

  (* Now that the arrival is handled, schedule the next arrival *)
     jetzt
  nextArrivalTime = simulationClock + drawWeibullNumber[λ];
  addEventToEventList[{nextArrivalTime, KINDARRIVAL}];
 ];
```

In[9]:=
```
(* Function that processes a departure event *)
handleDepartureEvent[departureTime_] := Module[{nextServiceTime},
   (* Check whether there is a boat queued up.  *)
   If[Length[queue] > 0,
     (* There is a boat
      queued: put this boat into service and schedule departure *)
     nextServiceTime = drawNormalNumber[meanServiceTime,
       varianceOfServiceTime, leftTruncation, rightTruncation];

     arrivalTimeOfBoatToBePutIntoService = removeAndReturnNextBoatFromQueue[];
     waitingTimeOfBoat =
      (simulationClock - arrivalTimeOfBoatToBePutIntoService) + nextServiceTime;
     totalWaitingTimeOfBoats += waitingTimeOfBoat;
     longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, waitingTimeOfBoat];
     addEventToEventList[{simulationClock + nextServiceTime, KINDDEPARTURE}];
     ,

     (* There is no customer queued: make the clerk available again *)
     currentNumberOfCranesBusy -= 1;
    ];
  ];
```

In[10]:=
```
(* Function that processes a fast departure event *)
handleFastDepartureEvent[departureTime_] := Module[{nextServiceTime},

  (* Checks whether there is a boat queued up *)
  If[Length[queue] > 0,
    (* If there is at least one boat in the queue,
    the boat is scheduled with normal service time even though two berths are
     free after the fast departure of one boat unloaded with two cranes *)
    nextServiceTime = drawNormalNumber[meanServiceTime,
      varianceOfServiceTime, leftTruncation, rightTruncation];

    arrivalTimeOfBoatToBePutIntoService = removeAndReturnNextBoatFromQueue[];
    waitingTimeOfBoat =
     (simulationClock - arrivalTimeOfBoatToBePutIntoService) + nextServiceTime;
    totalWaitingTimeOfBoats += waitingTimeOfBoat;
    longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, waitingTimeOfBoat];
    addEventToEventList[{simulationClock + nextServiceTime, KINDDEPARTURE}];

    (* Check whether there is another boat queued up *)
    If[Length[queue] > 0,
     (* There is another boat
      queued: put it into service and schedule departure *)
     nextServiceTime = drawNormalNumber[meanServiceTime,
       varianceOfServiceTime, leftTruncation, rightTruncation];

     arrivalTimeOfBoatToBePutIntoService = removeAndReturnNextBoatFromQueue[];
     waitingTimeOfBoat =
      (simulationClock - arrivalTimeOfBoatToBePutIntoService) + nextServiceTime;
     totalWaitingTimeOfBoats += waitingTimeOfBoat;
     longestWaitingTimeOfBoats =
      Max[longestWaitingTimeOfBoats, waitingTimeOfBoat];

     addEventToEventList[{simulationClock + nextServiceTime, KINDDEPARTURE}];

     ,
     currentNumberOfCranesBusy -= 1;
    ];
    ,

    (* There is no boat queued: make both cranes available again *)
    currentNumberOfCranesBusy -= 2;
   ];
 ];
```

In[11]:=
```
(* Function that takes the next event from the event list and processes it. *)
handleNextEvent[] := Module[{nextEvent, eventTime, eventKind},
                                nextEvent = removeAndReturnNextEventFromList[];
                                eventTime = nextEvent[[1]];
                                eventKind = nextEvent[[2]];

                                (* Update counters and parameters *)
                                simulationClock = eventTime;

                                (* Checks whether the next event
    is an arrival or a departure and calls functions accordingly *)
                                If[eventKind == KINDARRIVAL ,
                                     handleArrivalEvent[eventTime];
                                 ,
                                     If[eventKind == KINDDEPARTURE,
                                       handleDepartureEvent[eventTime];,

                                       handleFastDepartureEvent[eventTime];
                                       ];
                                 ];
                           ];
```

In[12]:=
```
(* Function that reports the input parameters
   Funktion
 and the findings of the simulation program *)
report[] := Module[{},
            Modul
                          Print["========================================"];
                          gib aus
                          Print["=== Simulation run of a harbor example ==="];
                          gib aus
                          Print["========================================"];
                          gib aus
                          Print["Input parameters:"];
                          gib aus  Eingabe
                          Print["Number of cranes c equals ", c, "."];
                          gib aus  Zahl

  Print["Mean interarrival time λ equals ", λ, " per day."];
  gib aus  arithmetisches Mittel
                          Print["Lower bound of service time ~ N(1,0.1) is ",
                          gib aus                                  numerischer Wert
   leftTruncation, " day(s)."];
                          Print["Upper bound of service time N(1,0.1}) is ",
                          gib aus                                  numerischer Wert
   rightTruncation, " day(s)."];
                          Print["========================================"];
                          gib aus
                          Print["Results:"];
                          gib aus

  Print["Duration of the simulation is ", simulationClock, " days"];
  gib aus  Dauer
                          Print["The total number of boats that arrived equals ",
                          gib aus
   totalNumberOfArrivals, "."];
                          Print["The average waiting time of boats equals ",
                          gib aus
   totalWaitingTimeOfBoats / totalNumberOfArrivals, " day(s)."];
                          Print["The longest waiting time of boats equals ",
                          gib aus
   longestWaitingTimeOfBoats, " day(s)."];
              ];
```

In[13]:=
```
(* Main Function *)
runProgram[] := Module[{averageWaitingTime},
                       preliminaries[];
                       initialise[];

   (* Handle events when the time of the earliest upcoming event,
   eventList[[1]][[1]], occurs before the end of the day,
   marked by numberOfHoursInADay *)
                       While[eventList[[1]][[1]] < numberOfDays,

                             handleNextEvent[];

                        ];
                       report[];

   averageWaitingTime = totalWaitingTimeOfBoats / totalNumberOfArrivals;
                       {averageWaitingTime, longestWaitingTimeOfBoats}
              ];
```

In[14]:=
```
(* Actually run the program: *)
runProgram[];
```

```
=======================================
=== Simulation run of a harbor example ===
=======================================
Input parameters:
Number of cranes c equals 2.
Mean interarrival time λ equals 1.25 per day.
Lower bound of service time ~ N(1,0.1) is 0.5 day(s).
Upper bound of service time N(1,0.1}) is 1.5 day(s).
=======================================
Results:
Duration of the simulation is 89.9639 days
The total number of boats that arrived equals 61.
The average waiting time of boats equals 0.774873 day(s).
The longest waiting time of boats equals 1.81317 day(s).
```

In[15]:=
```
(* Checks result consistency by replication to check simulation reliability*)
n = 100;
simulationResults = Table[runProgram[], {i, n}];
```

In[17]:=
```
(*Extract the first and second elements from each sub-list*)
  extrahiere
averageWaitingTimes = First /@ simulationResults;
                      erstes Element
longestWaitingTimes = simulationResults[[All, 2]];
                                        alle

(*Calculate the average of the first elements*)
averageOfAverageWaitingTimes = Mean[averageWaitingTimes]
                               arithmetisches Mittel
varianceOfAverageWaitingTimes = Variance[averageWaitingTimes]
                                Varianz

averageOfLongestWaitingTimes = Mean[longestWaitingTimes]
                               arithmetisches Mittel
varianceOfLongestWaitingTimes = Variance[longestWaitingTimes]
                                Varianz
```

Out[19]= 0.849638

Out[20]= 0.00730321

Out[21]= 2.15093

Out[22]= 0.132015